WhatWeEatInAmerica

January 12, 2026

1 Stratifying "What we eat in America"

We apply Dleto stratification on tensors emerging from nutrition data. We use a USDA Government survey

> What We Eat in America, NHANES 2017-March 2020 Prepandemic

The data consists of several thousand foods sold to the public within the USA with each item assigned measures of multiple food types. For example, what fraction of a serving consists of real fruits, vegetables, grains, added sugars and etc.

> **DISCLAIMER.** This notebook is intended to illustrate Dleto stratification algorithms only and does not represent any suggested or implied nutritional advise nor does it represent a complete understanding of the underlying significance of the data.

1.1 Loading the data

We begin by loading the data and preparing for analysis.

Let us start by loading the package we will use. In some situations you may need to add the package to your Julia system by uncommenting the relevant commands. This is typically a one-time step and can be avoided in future runs.

[1]:
```
using Pkg
# Uncomment to install packages if needed.
# Pkg.add("CSV"); Pkg.add("DataFrames")
using CSV, DataFrames
Pkg.activate("../..")  # Activate the main project
Pkg.instantiate()  # Ensure all dependencies are installed
Pkg.update()
using Dleto
using Plots
```

```
  Activating project at `~/CODE/OpenDleto`
    Updating registry at `~/.julia/registries/General.toml`
   Installed StatsBase   v0.34.10
   Installed NDTensors   v0.4.18
   Installed ITensors   v0.9.17
    Updating `~/CODE/OpenDleto/Project.toml`
  [9136182c] ↑ ITensors v0.9.15  v0.9.17
    Updating `~/CODE/OpenDleto/Manifest.toml`
  [9136182c] ↑ ITensors v0.9.15  v0.9.17
  [23ae76d9] ↑ NDTensors v0.4.17  v0.4.18
  [2913bbd2] ↑ StatsBase v0.34.9  v0.34.10
Precompiling packages…
   1426.4 ms   StatsBase
   2210.8 ms   NDTensors
   3443.5 ms   ITensors
   1199.2 ms   ITensors → ITensorsVectorInterfaceExt
  17840.9 ms   Plots
   1358.1 ms   Dleto
   1281.0 ms   Dleto → DletoIterativeSolversExt
   1485.1 ms   Dleto → DletoKrylovKitExt
   3537.2 ms   Dleto → DletoPlotsExt
  9 dependencies successfully precompiled in 24 seconds. 307 already
precompiled.
        Info We haven't cleaned this depot up for a bit,
running Pkg.gc()…
      Active manifest files: 6 found
      Active artifact files: 110 found
      Active scratchspaces: 4 found
     Deleted 2 package installations (735.711 KiB)
     Deleted 1 scratchspace (0 bytes)

WebIO._IJuliaInit()

Loading Dleto Plots Extension

[ Info: Precompiling IJuliaExt
[2f4121a4-3b3a-5ce6-9c5e-1f2673ce168a] (cache misses: wrong dep version loaded
(4))


SYSTEM: caught exception of type :MethodError while trying to print a failed
Task notice; giving up
```

Note. The USDA data set is provided as a single Microsoft Excell formated spreadsheet consisting of two sheets, the data, and a key. For convenience the file is stored locally as two separate CSV files, one for each sheat.

[2]:
```
# Load the data and key files
data = CSV.read("FPED_1720.csv", DataFrame)
key_data = CSV.read("FPED_1720-key.csv", DataFrame)
```

```
println("Total numer of foods: $(size(data,1))")
println("Number of Categories: $(size(key_data,1))")
println("\nSample of food items:")
println(data[147:155, vcat(1:2, 33:34, 38)])  # Show first few items and columns
```

```
Total numer of foods: 7444
Number of Categories: 39

Sample of food items:
9×5 DataFrame
 Row   FOODCODE  DESCRIPTION
D_MILK (cup eq)  D_YOGURT (cup eq)  ADD_SUGARS (tsp eq)
       Int64     String
 Float64          Float64            Float64



   1   11513802  Chocolate milk, made from light …            0.36
0.0                  0.85
   2   11513803  Chocolate milk, made from light …            0.36
0.0                  0.85
   3   11513804  Chocolate milk, made from light …            0.36
0.0                  0.85
   4   11513805  Chocolate milk, made from light …            0.18
0.0                  1.61
   5   11513850  Chocolate milk, made from sugar …            0.36
0.0                  0.0
   6   11513851  Chocolate milk, made from sugar …            0.36
0.0                  0.0
   7   11513852  Chocolate milk, made from sugar …            0.36
0.0                  0.0
   8   11513853  Chocolate milk, made from sugar …            0.36
0.0                  0.0
   9   11513854  Chocolate milk, made from sugar …            0.36
0.0                  0.0
```

If we need more detail about the labels we can inspect them individually. For example it may be clear from the food product that includes "light syrup" that sugar is an added ingredient where as "sugar free syrup" add 0.0 sugar.

[3]:
```
println( data[147,2])
println( data[155,2])
```

```
Chocolate milk, made from light syrup with reduced fat milk
Chocolate milk, made from sugar free syrup with fat free milk
```

To understand the 39 categories we can access the `key_data`.

[4]:
```
# Examine the key data to understand the nutritional categories
println("Key data structure:")
println(key_data)
```

```
Key data structure:
39×4 DataFrame
 Row   column   variable_name
variable_description                 units
      String3  String31          String
 String15?


   1   A        FOODCODE          Food code
missing
   2   B        DESCRIPTION       Food description
missing
   3   C        F_TOTAL           Total intact or cut fruits and f…  cup eq
   4   D        F_CITMLB          Intact fruits (whole or cut) of …  cup eq
   5   E        F_OTHER           Intact fruits (whole or cut); ex…  cup eq
   6   F        F_JUICE           Fruit juices, citrus and non cit…  cup eq
   7   G        V_TOTAL           Total dark green, red and orange…  cup eq
   8   H        V_DRKGR           Dark green vegetables              cup eq
   9   I        V_REDOR_TOTAL     Total red and orange vegetables …  cup eq
  10   J        V_REDOR_TOMATO    Tomatoes and tomato products       cup eq
  11   K        V_REDOR_OTHER     Other red and orange vegetables,…  cup eq
  12   L        V_STARCHY_TOTAL   Total starchy vegetables (white …  cup eq
  13   M        V_STARCHY_POTATO  White potatoes                    cup eq
  14   N        V_STARCHY_OTHER   Other starchy vegetables, exclud…  cup eq
  15   O        V_OTHER           Other vegetables not in the vege…  cup eq
  16   P        V_LEGUMES         Legumes computed as vegetables     cup eq
  17   Q        G_TOTAL           Total whole and refined grains     oz eq
  18   R        G_WHOLE           Whole grains                       oz eq
  19   S        G_REFINED         Refined or non-whole grains        oz eq
  20   T        PF_TOTAL          Total meat, poultry, seafood, or…  oz eq
  21   U        PF_MPS_TOTAL      Total meat, poultry, seafood, or…  oz eq
  22   V        PF_MEAT           Beef, veal, pork, lamb, game mea…  oz eq
  23   W        PF_CUREDMEAT      Cured/luncheon meat made from be…  oz eq
  24   X        PF_ORGAN          Organ meat from beef, veal, pork…  oz eq
  25   Y        PF_POULT          Chicken, turkey, Cornish hens, a…  oz eq
  26   Z        PF_SEAFD_HI       Seafood (finfish, shellfish and …  oz eq
  27   AA       PF_SEAFD_LOW      Seafood (finfish, shellfish and …  oz eq
  28   AB       PF_EGGS           Eggs (chicken, duck, goose, quai…  oz eq
  29   AC       PF_SOY            Soy products, excluding calcium …  oz eq
  30   AD       PF_NUTSDS         Peanuts, tree nuts, and seeds, e…  oz eq
  31   AE       PF_LEGUMES        Legumes computed as protein foods  oz eq
```

```
  32   AF       D_TOTAL           Total milk, yogurt, cheese, and …  cup eq
  33   AG       D_MILK            Fluid milk and calcium fortified…  cup eq
  34   AH       D_YOGURT          Yogurt                             cup eq
  35   AI       D_CHEESE          Cheese                             cup eq
  36   AJ       OILS              Oils                               grams
  37   AK       SOLID_FATS        Solid fats                         grams
  38   AL       ADD_SUGARS        Foods defined as added sugars      tsp eq
  39   AM       A_DRINKS          Alcoholic beverages                no. of
drinks
```

We also do an initial analysis to identify missing values and total range of values to prepare for further analysis.

[5]:
```
using Statistics

# Extract the numerical data for tensor creation
# Remove the first two columns (FOODCODE and DESCRIPTION) to get only numerical␣
 ↪data
numerical_data = Matrix(data[:, 3:end])

println("Numerical data shape: $(size(numerical_data))")
println("Data type: $(eltype(numerical_data))")

# Check for any missing values
missing_count = sum(ismissing.(numerical_data))
println("Missing values: $missing_count")

# Summary statistics
println("\nData range:")
println("Min value: $(minimum(skipmissing(numerical_data)))")
println("Max value: $(maximum(skipmissing(numerical_data)))")
println("Mean value: $(mean(skipmissing(numerical_data)))")

# Check if data is sparse (many zeros)
zero_count = sum(numerical_data .== 0)
total_elements = length(numerical_data)
sparsity = zero_count / total_elements
println("Sparsity (fraction of zeros): $(round(sparsity, digits=3))")
```

```
Numerical data shape: (7444, 37)
Data type: Float64
Missing values: 0

Data range:
Min value: 0.0
Max value: 100.0
Mean value: 0.329505351670854
Sparsity (fraction of zeros): 0.831
```

So wee that this data is carefully constructed and actually has no missing values. Yet, many values are 0.0 which may indicate those measurements are below a threshold.

1.2 Creating food Categories

We will now create an analysis of this data set which explores general categories of nutrition as supplied in the What We Eat in America Food Categories. Of the 15 categories identified ther, we simplify the study into the following 7 slightly more general categories. - FRUITS: citrus/melons/berries, other fruits, and fruit juice - VEGETABLES: color-based groupings (dark green, red/orange), starchy vegetables (potatoes vs others), legumes, and other vegetables - GRAINS: whole grains and refined grains - PROTEIN FOODS: meat, poultry, seafood (high/low omega-3), eggs, soy, nuts/seeds, and legumes - DAIRY: milk, yogurt, and cheese - FATS & OILS: liquid oils and solid fats - DISCRETIONARY CALORIES: added sugars and alcoholic beverages

An inspection of the column headings above shows the fruit related columns are indexed by `F_` terms, vegetables by `V_`, grains by `G_` and so forth. However, there is also a cummualtive column `F_Total` which we may exclude as it is derived from the other columns.

So we create these categories for our computation.

[6]:
```
nutritional_columns = names(data)[3:end]  # Skip FOODCODE and DESCRIPTION

# Define natural category groupings based on USDA food patterns
isfruit(col) = startswith(string(col), "F_") && !occursin("TOTAL", string(col))
fruits = filter(isfruit, nutritional_columns)

println("\n FRUITS ($(length(fruits)) categories):")
for (i, fruit) in enumerate(fruits)
    println("  $i. $fruit")
end
```

```
 FRUITS (3 categories):
  1. F_CITMLB (cup eq)
  2. F_OTHER (cup eq)
  3. F_JUICE (cup eq)
  2. F_OTHER (cup eq)
  3. F_JUICE (cup eq)
```

[7]:
```
isvegetable(col) = startswith(string(col), "V_") && !occursin("TOTAL",␣
 ↪string(col))
vegetables = filter(isvegetable, nutritional_columns)
println("\n VEGETABLES ($(length(vegetables)) categories):")
for (i, veg) in enumerate(vegetables)
    println("  $i. $veg")
end
```

```
 VEGETABLES (7 categories):
  1. V_DRKGR (cup eq)
```

```
 2. V_REDOR_TOMATO (cup eq)
 3. V_REDOR_OTHER (cup eq)
 4. V_STARCHY_POTATO (cup eq)
 5. V_STARCHY_OTHER (cup eq)
 6. V_OTHER (cup eq)
 7. V_LEGUMES (cup eq)
 2. V_REDOR_TOMATO (cup eq)
 3. V_REDOR_OTHER (cup eq)
 4. V_STARCHY_POTATO (cup eq)
 5. V_STARCHY_OTHER (cup eq)
 6. V_OTHER (cup eq)
 7. V_LEGUMES (cup eq)
```

[8]:
```
isgrain(col) = startswith(string(col), "G_") && !occursin("TOTAL", string(col))
grains       = filter(isgrain, nutritional_columns)
println("\n GRAINS ($(length(grains)) categories):")
for (i, grain) in enumerate(grains)
    println("  $i. $grain")
end
```

```
 GRAINS (2 categories):
 1. G_WHOLE (oz eq)
 2. G_REFINED (oz eq)
```

[9]:
```
isprotein(col) = startswith(string(col), "PF_") && !occursin("TOTAL",␣
 ↪string(col))
proteins = filter(isprotein, nutritional_columns)
println("\n PROTEIN FOODS ($(length(proteins)) categories):")
for (i, protein) in enumerate(proteins)
    println("  $i. $protein")
end
```

```
 PROTEIN FOODS (10 categories):
 1. PF_MEAT (oz eq)
 2. PF_CUREDMEAT (oz eq)
 3. PF_ORGAN (oz eq)
 4. PF_POULT (oz eq)
 5. PF_SEAFD_HI (oz eq)
 6. PF_SEAFD_LOW (oz eq)
 7. PF_EGGS (oz eq)
 8. PF_SOY (oz eq)
 9. PF_NUTSDS (oz eq)
 10. PF_LEGUMES (oz eq)
```

[10]:
```
isdairy(col) = startswith(string(col), "D_") && !occursin("TOTAL", string(col))
dairy        = filter(isdairy, nutritional_columns)
```

```
println("\n  DAIRY ($(length(dairy)) categories):")
for (i, d) in enumerate(dairy)
    println("  $i. $d")
end
```

```
 DAIRY (3 categories):
  1. D_MILK (cup eq)
  2. D_YOGURT (cup eq)
  3. D_CHEESE (cup eq)
```

[11]:
```
isfat_or_oil(col) = string(col) in ["OILS (grams)", "SOLID_FATS (grams)"]
fats_oils     = filter(isfat_or_oil, nutritional_columns)
println("\n  FATS & OILS ($(length(fats_oils)) categories):")
for (i, fat) in enumerate(fats_oils)
    println("  $i. $fat")
end
```

```
 FATS & OILS (2 categories):
  1. OILS (grams)
  2. SOLID_FATS (grams)
```

[12]:
```
isdiscretionary(col) = string(col) in ["ADD_SUGARS (tsp eq)", "A_DRINKS (no. of␣
 ↪drinks)"]
discretionary = filter(isdiscretionary, nutritional_columns)
println("\n  DISCRETIONARY CALORIES ($(length(discretionary)) categories):")
for (i, disc) in enumerate(discretionary)
    println("  $i. $disc")
end
```

```
 DISCRETIONARY CALORIES (2 categories):
  1. ADD_SUGARS (tsp eq)
  2. A_DRINKS (no. of drinks)
  1. ADD_SUGARS (tsp eq)
  2. A_DRINKS (no. of drinks)
```

[13]:
```
println("SUMMARY OF THE 7 NUTRITIONAL CATEGORIES:")
println("="^50)
println("  FRUITS ($(length(fruits))):")
println("  - Citrus/melons/berries, other fruits, fruit juice (excluding␣
 ↪totals)")

println("\n  VEGETABLES ($(length(vegetables))):")
println("  - Dark green, red/orange, starchy (potatoes/others), legumes, other␣
 ↪vegetables (excluding totals)")
```

```
println("\n  GRAINS ($(length(grains))):")
println("   - Whole grains and refined grains (excluding totals)")

println("\n  PROTEIN FOODS ($(length(proteins))):")
println("   - Meat, poultry, seafood (high/low omega-3), eggs, soy, nuts/seeds,␣
 ↪legumes (excluding totals)")

println("\n  DAIRY ($(length(dairy))):")
println("   - Milk, yogurt, cheese (excluding totals)")

println("\n  FATS & OILS ($(length(fats_oils))):")
println("   - Liquid oils and solid fats")

println("\n  DISCRETIONARY CALORIES ($(length(discretionary))):")
println("   - Added sugars and alcoholic beverages")
```

```
SUMMARY OF THE 7 NUTRITIONAL CATEGORIES:
================================================
  FRUITS (3):
   - Citrus/melons/berries, other fruits, fruit juice (excluding totals)

  VEGETABLES (7):
   - Dark green, red/orange, starchy (potatoes/others), legumes, other vegetables
(excluding totals)

  GRAINS (2):
   - Whole grains and refined grains (excluding totals)

  PROTEIN FOODS (10):
   - Meat, poultry, seafood (high/low omega-3), eggs, soy, nuts/seeds, legumes
(excluding totals)

  DAIRY (3):
   - Milk, yogurt, cheese (excluding totals)

  FATS & OILS (2):
   - Liquid oils and solid fats

  DISCRETIONARY CALORIES (2):
   - Added sugars and alcoholic beverages
```

1.3 A "What-We-Eat" Tensor.

There are many ways to define a tensor from these data. As illustration will will explore the relationship of added sugars to products that range over the categories of Fruits, Vegetables, and Proteins. This will prescribe a 3-way tensor with axes `Fruit x Vegetables x Protein` where very entry is the total amount of added sugars for food groups in that category.

As a technical matter, since the data is very sparse there will be many entries that score `0.0` in one or more of the axes categories. In that case we add a special `Empty` class to each of these categories to indicate that the food group does not contain any measureable amount of the specified category. For example, milk main contain `0.0` detectable Fruit so it would be placed in the `Empty` fruit category.

[14]:
```
# Use the specific categories but add "empty" options
fruits_with_empty = vcat(fruits, ["F_EMPTY"])
vegetables_with_empty = vcat(vegetables, ["V_EMPTY"])
proteins_with_empty = vcat(proteins, ["PF_EMPTY"])

println("(#Fruits, #Vegetables, #Proteins) = ($(length(fruits_with_empty)),␣
 ↪$(length(vegetables_with_empty)), $(length(proteins_with_empty)) )")
```

```
(#Fruits, #Vegetables, #Proteins) = (4, 8, 11 )
```

Now we create the tensor with these axes and total added sugar as the value in each entry. We will be using `ITensors` as our backend for tensors. These require that each axis be created as an `Index` which will prevent the complication of remembering which axis was first, second, or third.

[15]:
```
using ITensors

# Create new tensor with empty categories
f = Index(length(fruits_with_empty), "Fruits")
v = Index(length(vegetables_with_empty), "Veg")
p = Index(length(proteins_with_empty), "Prot")

println("\nIndices: $(f) × $(v) × $(p)")
```

```
Indices: (dim=4|id=412|"Fruits") × (dim=8|id=93|"Veg") × (dim=11|id=175|"Prot")
```

To create the tensor we traverse very row of the data and check for a nonzero added sugar value. Upon encountering such a value we then designate the `(f,v,p)` coordinates of that entry add them to any existing value in the coordinate.

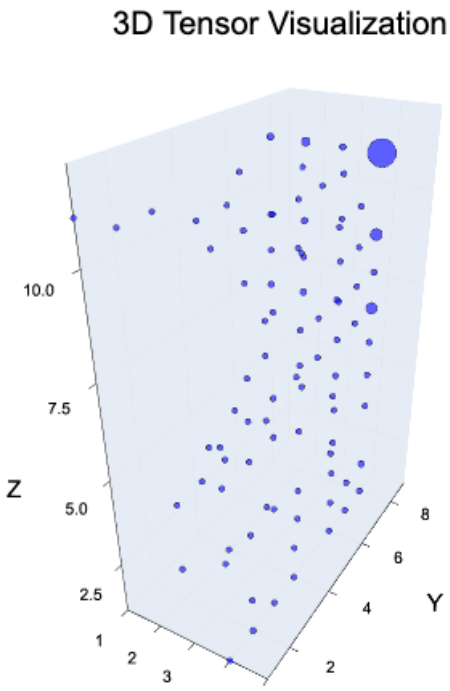
[16]:
```
Sugar = ITensor(f,v,p)

processed_foods = 0
empty_assignments = Dict("fruits" => 0, "vegetables" => 0, "proteins" => 0)

for row_idx in 1:nrow(data)
    sugar_value = data[row_idx, "ADD_SUGARS (tsp eq)"]

    if ismissing(sugar_value) || sugar_value == 0.0
        continue
    end

    # Get vector of fruit, veg, and protein for this food entry.
```

```
    fruit_vals = [coalesce(data[row_idx, col], 0.0) for col in fruits]
    veg_vals = [coalesce(data[row_idx, col], 0.0) for col in vegetables]
    protein_vals = [coalesce(data[row_idx, col], 0.0) for col in proteins]

    # Determine indices with empty categories
    if maximum(fruit_vals) <= 1e-10  # Essentially zero
        fruit_idx = length(fruits_with_empty)  # Use empty category
        empty_assignments["fruits"] += 1
    else
        fruit_idx = argmax(fruit_vals)
    end

    if maximum(veg_vals) <= 1e-10  # Essentially zero
        veg_idx = length(vegetables_with_empty)  # Use empty category
        empty_assignments["vegetables"] += 1
    else
        veg_idx = argmax(veg_vals)
    end

    if maximum(protein_vals) <= 1e-10  # Essentially zero
        protein_idx = length(proteins_with_empty)  # Use empty category
        empty_assignments["proteins"] += 1
    else
        protein_idx = argmax(protein_vals)
    end

    # Add sugar value to tensor
    Sugar[f=>fruit_idx, v=>veg_idx, p=>protein_idx] += sugar_value
    processed_foods += 1
end

# Analyze the new tensor with empty categories
T_empty_array = Array(Sugar, f, v, p)
non_zero_empty = count(x -> x != 0, T_empty_array)

println("\n Tensor with empty categories created!")
println("Foods with added sugars: $processed_foods, $(round((processed_foods /␣
 ↪(size(data,1)-2)) * 100, digits=2))%")
println("Non-zero entries: $non_zero_empty / $(length(T_empty_array))")
println("Sparsity: $(round((length(T_empty_array) - non_zero_empty) /␣
 ↪length(T_empty_array) * 100, digits=1))%")
println("Max value: $(round(maximum(T_empty_array), digits=2)) tsp eq")

println("  Foods with 0 fruits: $(empty_assignments["fruits"]) foods")
println("  Foods with 0 vegetables: $(empty_assignments["vegetables"]) foods")
println("  Foods with 0 proteins: $(empty_assignments["proteins"]) foods")
```

```
  Tensor with empty categories created!
Foods with added sugars: 3194, 42.92%
Non-zero entries: 91 / 352
Sparsity: 74.1%
Max value: 4210.98 tsp eq
  Foods with 0 fruits: 2708 foods
  Foods with 0 vegetables: 2471 foods
  Foods with 0 proteins: 1424 foods
```

[17]:
```
plot_tensor(Sugar)
```

[17]:



We see that this data indicates that 42% of the foods consumed in the USA for this study contained added sugars. We now will look to Dleto's `startify` operations to see if we can detect the types of products that use the most added sugars.

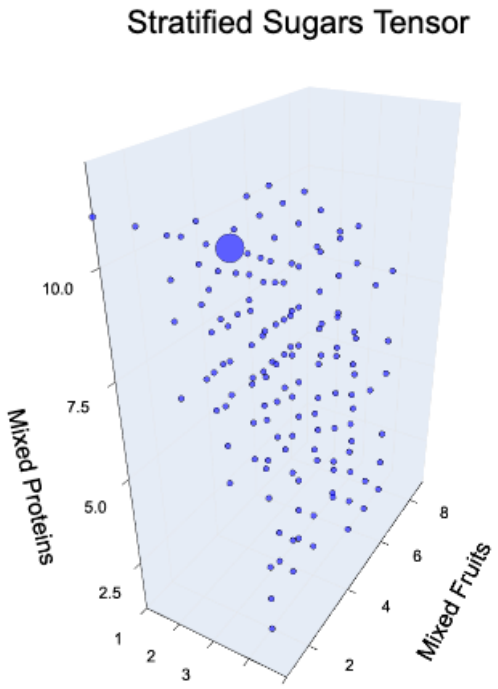
[18]:
```
# Apply Dleto tensor analysis: nondeg and stratify
using Dleto

println("TENSOR STRATIFICATION ANALYSIS")
println("="^50)

# # First apply nondeg to create nondegenerate tensor
```

```
# println("Applying nondeg()...")
# @time Sugar_nd, Xs_nd = nondeg(Sugar)

# println("\n  Nondegenerate tensor created.")
# println("Original tensor dimensions: $(dims(Sugar))")
# println("Nondegenerate tensor dimensions: $(dims(Sugar_nd))")

# Apply stratify to find stratified structure
println("\nAttempting stratification...")
@time Sugar_strat, Xs_strat = stratify(Sugar);
```

```
TENSOR STRATIFICATION ANALYSIS
================================================

Attempting stratification…

[ Info: Found 7 derivations for
stratification.

  4.907966 seconds (45.60 M allocations: 2.200 GiB, 8.10% gc time, 99.66%
compilation time)
```

[19]:
```
plot_tensor(Sugar_strat, title="Stratified Sugars Tensor",
        xlabel="Mixed Veg.", ylabel="Mixed Fruits", zlabel="Mixed Proteins")
```

[19]:



1.4 Analysizing Stratification

If we take a minute to inspect our stratified sugar tensor we may notice there is a corner of the plot where a row of dots is separated from the larger cluster. This is most noticeable if we look ath the `(Proteins,Fruits)` face of the 3D plot. To understand the cluster we must reconstruct the values in terms of the original coordiantes. This is done by inspecting the columns of the transverse operator `Xs_st` returned during stratification.

As a technical note we may be concerned about matching the correct axes, for example, what if `Xs_st[1]` corresponds to Fruit but we misstook it as protein? Fortunately the transformations are attached to their indices so the label, not the order, is what matters. We can test this before exploring our data.

[20]:
```
inds(Xs_strat[1])
```

[20]:
```
((dim=4|id=412|"Fruits"), (dim=4|id=878|"(dim=4|id=412|"F,Site:Der"))
```

If the first index coincides with fruit then one of our indices will have the tag “Fruits”. The second index references the output after stratifying and it is a machine generated label relating to the Dleto infrastructure. The operations know this and will only apply coordinates to matching indices.

With this in mind, if we appeal the tensor plot of our stratified sugar tensor we might find the block coincides with the first coordinate along the new “Mixed Fruitsfruit axis. So let us explore that vector in its original fruit coordinates.

[21]:
```
# Create the standard "one-hot" vector e_1 for fruits
fruit_1 = ITensor( f ); fruit_1[f=>1] = 1.0;
# Compute the mix of fruits that replaced e_1.
mix_fruits_1 =  Xs_strat[1] * fruit_1
# Convert to array for convenience
mix_fruits_1 = Array( mix_fruits_1, inds(mix_fruits_1))

# Define fruit descriptions
fruit_descriptions = Dict(
    "F_CITMLB (cup eq)" => "Citrus, Tomatoes, Melons, and Berries",
    "F_OTHER (cup eq)" => "Other Fruits (apples, bananas, etc.)",
    "F_JUICE (cup eq)" => "Fruit Juices",
    "F_EMPTY" => "Foods with no significant fruit content"
)

# Print out the proportions of the mix.
println("Fruit coordinate composition:")
for (i, weight) in enumerate(mix_fruits_1)
    if abs(weight) > 1e-10
        fruit_name = fruits_with_empty[i]
        fruit_description = fruit_descriptions[fruit_name]
        println("  $(round(weight*100, digits=1))% × $fruit_description")
```

```
    end
end
```

```
Fruit coordinate composition:
  -98.6% × Citrus, Tomatoes, Melons, and Berries
  87.1% × Other Fruits (apples, bananas, etc.)
  -82.3% × Fruit Juices
  -78.4% × Foods with no significant fruit content
```

In hind sight perhaps few readers will surprising that added sugars are nearly always involved with citrus fruits, and 4/5 instances of fruit juices. However, we have not yet explored the protain combination that identifies this cluster. In our coordinates it is the largest protein index that isolates the cluster. So we reconstruct the protein mixture that identifies that coordinate.

Note can afford to guess the coordinate of `Xs_strat` that addresses protein as the indices cannot be applied to other axes. An error will force us to pick the correct corrdinate.

> **REMARK** In a future release we may replace such returns with a dictionary for automatic lookup.

[22]:
```
# Create the standard "one-hot" vector e_last for protein
protein_last = ITensor( p ); protein_last[p=>dim(p)] = 1.0;
# Compute the mix of proteins that replaced e_last.
mix_protein =  Xs_strat[3] * protein_last
# Convert to array for convenience
mix_protein = Array( mix_protein, inds(mix_protein))

protein_descriptions = Dict(
    "PF_MEAT (oz eq)" => "Meat (beef, pork, lamb, game)",
    "PF_CUREDMEAT (oz eq)" => "Cured/Processed Meats (bacon, sausage, deli)",
    "PF_ORGAN (oz eq)" => "Organ Meats (liver, kidney, etc.)",
    "PF_POULT (oz eq)" => "Poultry (chicken, turkey, duck)",
    "PF_SEAFD_HI (oz eq)" => "High Omega-3 Seafood (salmon, sardines)",
    "PF_SEAFD_LOW (oz eq)" => "Low Omega-3 Seafood (cod, shrimp)",
    "PF_EGGS (oz eq)" => "Eggs and Egg Products",
    "PF_SOY (oz eq)" => "Soy Products (tofu, tempeh, soy milk)",
    "PF_NUTSDS (oz eq)" => "Nuts and Seeds",
    "PF_LEGUMES (oz eq)" => "Legumes (beans, lentils, peas)",
    "PF_EMPTY" => "Foods with no significant protein content"
)

# Print out the proportions of the mix.
println("Protein coordinate composition:")
for (i, weight) in enumerate(mix_protein)
    if abs(weight) > 1e-10
        protein_name = proteins_with_empty[i]
        protein_description = get(protein_descriptions, protein_name,␣
 ↪protein_name)
        println("  $(round(weight*100, digits=1))% × $protein_description")
```

```
    end
end
```

```
Protein coordinate composition:
  -0.0% × Meat (beef, pork, lamb, game)
  -0.0% × Cured/Processed Meats (bacon, sausage, deli)
  0.0% × Organ Meats (liver, kidney, etc.)
  0.0% × Poultry (chicken, turkey, duck)
  0.0% × High Omega-3 Seafood (salmon, sardines)
  0.0% × Low Omega-3 Seafood (cod, shrimp)
  0.0% × Eggs and Egg Products
  0.1% × Soy Products (tofu, tempeh, soy milk)
  0.0% × Nuts and Seeds
  0.1% × Legumes (beans, lentils, peas)
  0.0% × Foods with no significant protein content
```

So this data tells the story that the must added sugar concerns foods comprised of little to no protein and either citrus or fruit juices.

At this point you are encouraged to revisit our construciton and create "What-We-Eat" tensors using different information. Perhaps replacing added sugar with "fats and oils" or comparing (dairy,grain,protein). While this data set may simply affirm patterns you already know, keep in mind how the stratfication algorithms modify the information to extract that information.

There is perhaps not much that would surprise us with that outcome. The surprise perhaps is that the chisel algorithms so clearly identify this pattern without any training nor understanding of nutrition. As with all data science projects, a subject matter expert will be a vital component of creating useful experiments for chiseling. The results are repeatable and perform well on data sets of this scale so they are just a further tool in the exploration of data.